

JavaFX常用控件之



# FlowPane和TilePane

---

北京理工大学计算机学院  
金旭亮

# FlowPane概述



FlowPane按照水平或垂直方向顺序排列子控件。



默认按照水平方向顺序排列子控件，可以调用以下方法更改排列方式为垂直方向：

```
FlowPane flowpane = new FlowPane(Orientation.VERTICAL);
```



子控件的大小可以有大有小，不强求一致。

# FlowPane的基础功能——“自动换行”



使用FlowPane，当窗体缩小时，会在下一行显示控件，即具备“自动换行”特性。

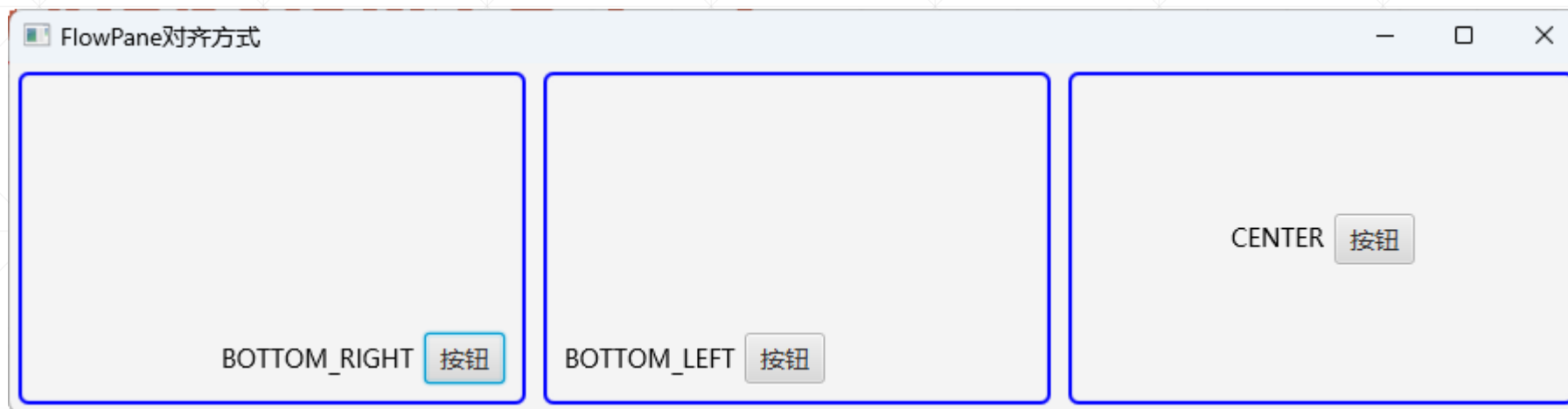
# 调整控件间距

为了排列好看，可以设置FlowPane的Hgap和Vgap，让控件之间保留一点空隙：

```
//设置水平和垂直间距
double hgap = 5;
double vgap = 10;
FlowPane root = new FlowPane(hgap, vgap);

for(int i = 1; i <= 10; i++) {
    root.getChildren().add(new Button(s: "按钮 " + i));
}
```

# FlowPane的对齐方式



使用`setPreferredSize`设定FlowPane的宽度和高度之后，可以进一步地调用`setAlignment`方法指定它所包容的控件的对齐方式，上面的示例给出了三种对齐方式。

# 设定对齐方式示例代码

```
private FlowPane createFlowPane(Pos alignment) {  
  
    FlowPane fp = new FlowPane( hgap: 5, vgap: 5);  
    fp.setPrefSize( prefWidth: 300, prefHeight: 200);  
    fp.setAlignment(alignment);  
    fp.getChildren().addAll(new Text(alignment.toString()),  
                             new Button( s: "按钮"));  
  
    fp.setStyle("-fx-padding: 10;" +  
               "-fx-border-style: solid inside;" +  
               "-fx-border-width: 2;" +  
               "-fx-border-insets: 5;" +  
               "-fx-border-radius: 5;" +  
               "-fx-border-color: blue;");  
  
    return fp;  
}
```

## Pos枚举类型常量值

TOP\_LEFT、TOP\_CENTER、  
TOP\_RIGHT

CENTER\_LEFT、CENTER、  
CENTER\_RIGHT

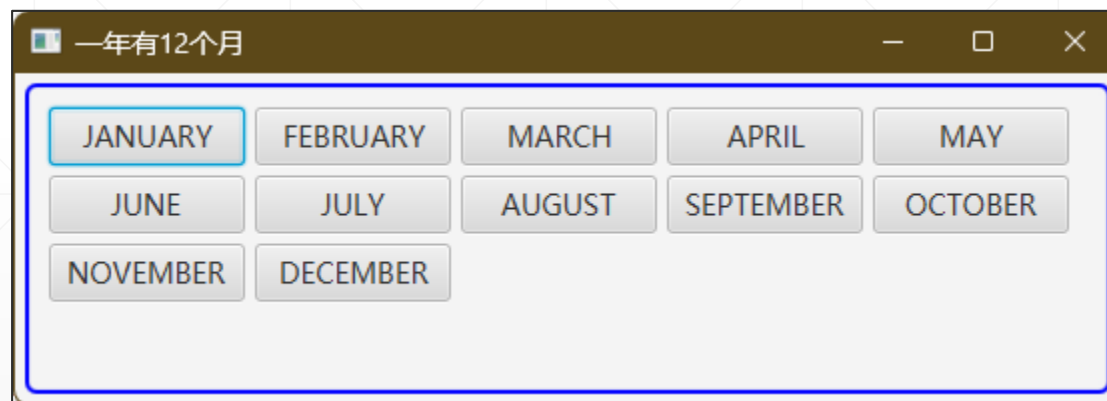
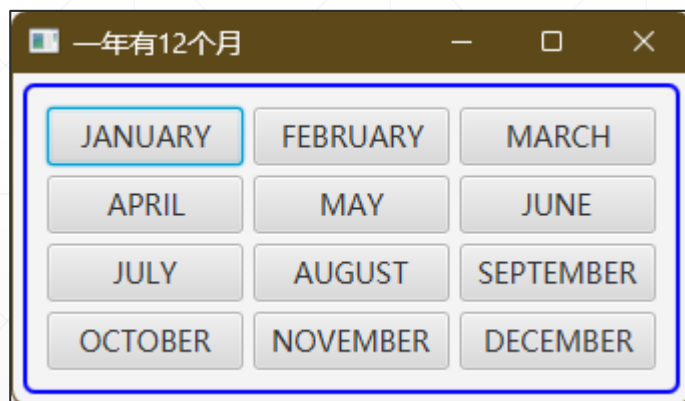
BOTTOM\_LEFT、BOTTOM\_CENTER、  
BOTTOM\_RIGHT

BASELINE\_LEFT、BASELINE\_CENTER、  
BASELINE\_RIGHT

# TilePane概述



TilePane功能与FlowPane基本一致，也是按行或按列排列多个控件，不同在于TilePane的子控件，其尺寸都是一样的。



由于TilePane的这个特点，比较适合于放置同一类控件或图形对象，让它们具有相同的高度和宽度，轻松地实现控件“整整齐齐”地排列。

# TilePane示例



```
double hgap = 5.0;
double vgap = 5.0;
TilePane root = new TilePane(hgap, vgap);
//设定每行显示3列
root.setPrefColumns(3);

// 12个月的英文, 有长有短
for(Month month: Month.values()) {
    Button b = new Button(month.toString());
    //让按钮自适应大小
    b.setMaxHeight(Double.MAX_VALUE);
    b.setMaxWidth(Double.MAX_VALUE);
    //加入到TilePane中
    root.getChildren().add(b);
}
```

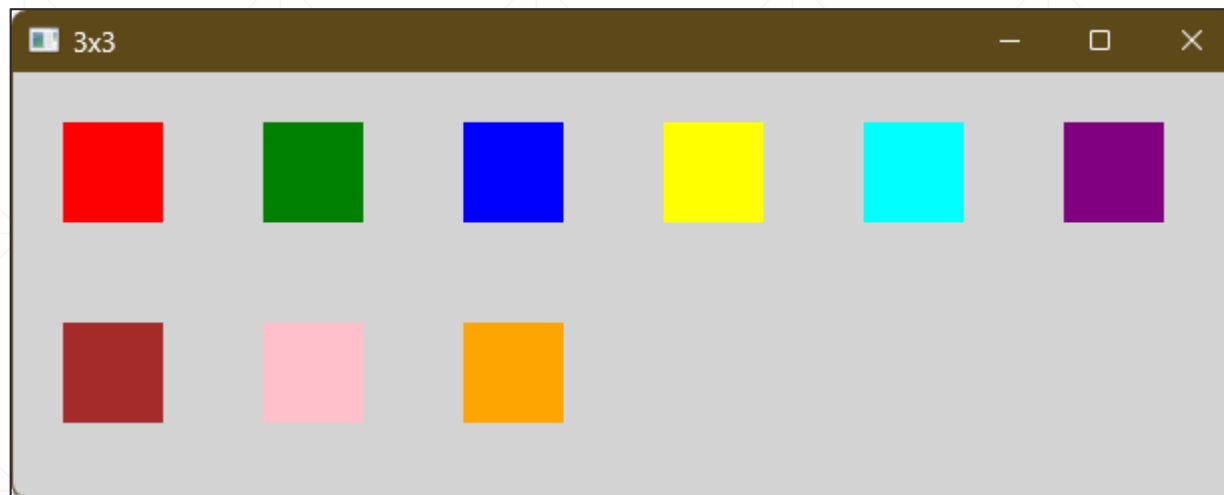
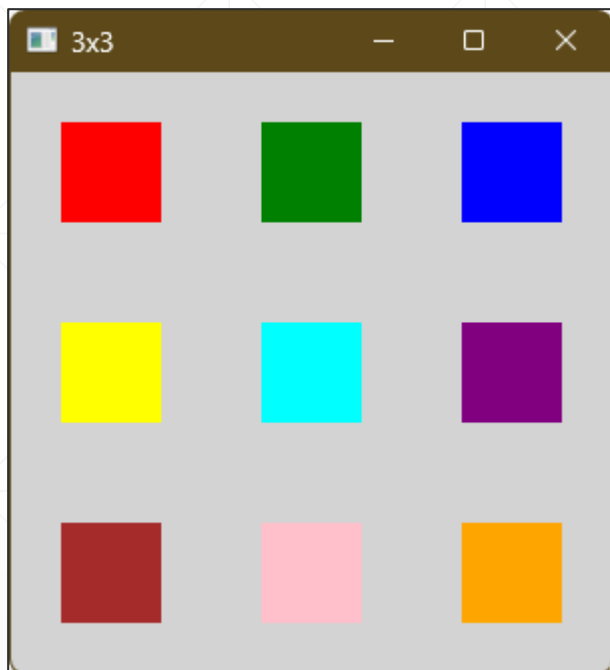
注意, TilePane只是为每个控件“留出了”一样大小的空间, 但它并不“强迫”所有子控件一定要占满这个空间。



因此, 你需要写代码让特定类型的子控件“自由伸展”, 以便占满所有空间。

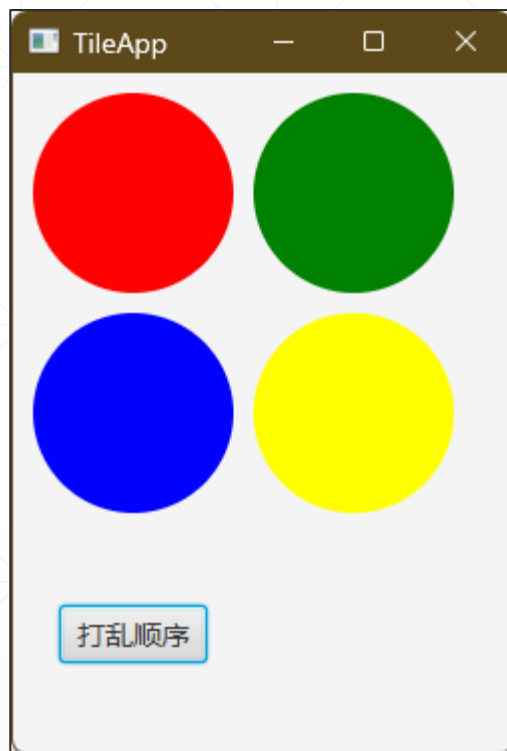


# 图形的规范排列

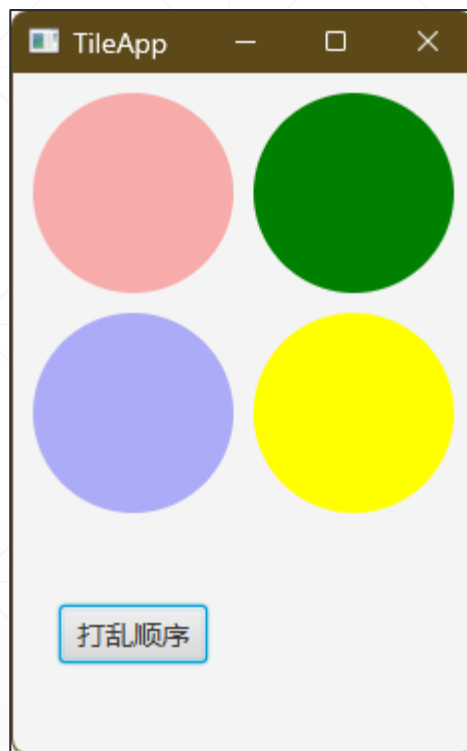


利用TilePane “自动等宽等高” 的特性，  
可以很容易地让多个图形 “规范排列”，  
同时又能在宽度改变时 “重新排列”。

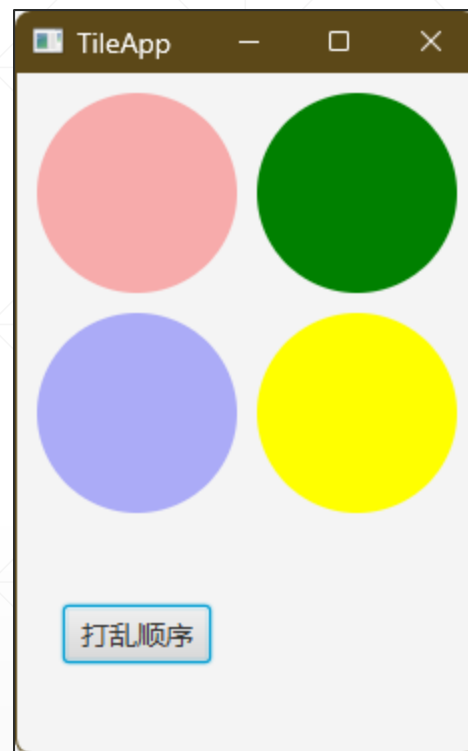
# TilePane综合示例



点击则  
“半透明”



切换顺序



改变窗体宽度



# 示例分析-1



//两行两列，自动居中

```
TilePane tilePane = new TilePane();  
tilePane.setPrefColumns(2);  
tilePane.setPrefRows(2);  
tilePane.setTileAlignment(Pos.CENTER);
```

实例化四个Circle对象，加入到集合中，然后遍历集合，为它们统一添加事件响应代码。

```
Circle redCircle = new Circle(radius: 50, Color.RED);  
Circle greenCircle = new Circle(radius: 50, Color.GREEN);  
Circle blueCircle = new Circle(radius: 50, Color.BLUE);  
Circle yellowCircle = new Circle(radius: 50, Color.YELLOW);
```

```
List<Circle> circles = new ArrayList<>();  
circles.add(redCircle);  
circles.add(greenCircle);  
circles.add(blueCircle);  
circles.add(yellowCircle);
```

```
circles.forEach(CirclesInTilePane::changeColorInClick);
```

## 示例分析-2



Shape有一个Properties属性可以用于保存一些“私有”数据。

```
private static void changeColorInClick(Circle c) {  
    c.setOnMouseClicked(e -> {  
        Boolean selected = (Boolean) c.getProperties().get("selected");  
        if (selected == null || selected == Boolean.FALSE) {  
            c.setOpacity(0.3d);  
            c.getProperties().put("selected", Boolean.TRUE);  
        } else {  
            c.setOpacity(1.0d);  
            c.getProperties().put("selected", Boolean.FALSE);  
        }  
    });  
}
```

## 示例分析-3



```
Button btn = new Button(s: "打乱顺序");  
btn.setOnAction(e -> {  
    Collections.shuffle(circles);  
    tilePane.getChildren().clear();  
    tilePane.getChildren().addAll(circles);  
    tilePane.getChildren().add(btn);  
});
```

```
tilePane.getChildren().addAll(circles);  
tilePane.getChildren().add(btn);  
tilePane.setHgap(10);  
tilePane.setVgap(10);  
tilePane.setPadding(new Insets(topRightBottomLeft: 10));
```

将集合打乱之后，重新加入到TilePane中。

设定TilePane中各个控件的间隔，及其与边界的距离。